

Semana 1 - Fuga para o Castelo

1. Identificação do Grupo

Integrante 1: Solano Omar Oliveira do Nascimento **NUSP 1:** 14608017

Integrante 2: Lays Vieira Zandomingos **NUSP 2:** 14587999

Integrante 3: Claudio Lucio Cunha da Silva **NUSP 3:** 14565003

Repositório: <https://github.com/claudiolucio/lab-proc/tree/main/projeto> — **Release:** v0.1.0

2. Leitura Pré-Aula

NYSTROM, R. *Game Programming Patterns*. Genever Benning, 2014. Disponível gratuitamente em gameprogrammingpatterns.com. Capítulos sobre os padrões Game Loop, State e Update Method. O padrão State resolve o problema central deste projeto: a alternância entre a fase de plataforma e o mundo de perguntas acessado pelos portais.

SOMMERVILLE, I. *Engenharia de Software*. 10. ed. Pearson (MinhaBiblioteca). Capítulos de engenharia de requisitos e de teste de software. Fundamenta a distinção entre requisitos funcionais e não-funcionais, a redação de requisitos verificáveis e a organização dos testes em níveis de módulo, integração e sistema.

PRESSMAN, R.; MAXIM, B. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. AMGH (MinhaBiblioteca). Capítulos de processo iterativo e incremental e de depuração, base do ciclo de trabalho da Seção 5.

CHACON, S.; STRAUB, B. *Pro Git*. 2. ed. Apress, 2014. Disponível em português em git-scm.com/book/pt-br/v2. Ramificação, fluxo distribuído e criação de tags. Complementado pela documentação do GitHub sobre Releases e pela especificação de Versionamento Semântico (semver.org).

Documentação oficial da biblioteca gráfica 2D adotada — tutoriais introdutórios de acesso público sobre laço principal, desenho de sprites, captura de teclado e colisão por retângulos delimitadores, recursos exigidos pelos requisitos RF01 a RF04.

Motivação do projeto. Jogos de plataforma tradicionais exploram quase somente a habilidade motora, desperdiçando o potencial educativo do meio; jogos puramente educativos, por sua vez, costumam falhar em manter o engajamento. *Fuga para o Castelo* busca o equilíbrio entre as duas dimensões. O jogador controla um príncipe que corre por um cenário lateral rumo ao castelo, saltando pedras e árvores. Ao longo do percurso encontra três portais que o transportam a um mundo de perguntas de conhecimentos gerais, onde deve responder a uma questão de múltipla escolha com alternativas de "a" a "d". O erro custa uma vida, o que dá consequência concreta à resposta e cria incentivo natural para que o jogador raciocine antes de decidir. O objetivo geral é desenvolver esse jogo 2D combinando destreza e conhecimento; os

objetivos específicos são implementar a movimentação e o salto do personagem, os obstáculos com detecção de colisão, o sistema de portais, o banco de perguntas, o gerenciamento de vidas e a condição de vitória. O público-alvo são estudantes e jogadores casuais de todas as idades.

3. Especificação de Requisitos e Testes

Especificação dos requisitos funcionais (RF) e não-funcionais (RNF) que o jogo deve atender e dos testes associados a cada requisito mapeado, conforme a Tabela 1. Por se tratar da entrega de planejamento da Semana 1, as colunas Resultado Obtido e Evidências de Resultados serão preenchidas conforme a implementação e a execução dos testes avançarem.

Requisito	RF ou RNF	Teste	Resultado Esperado	Resultado Obtido	Evidências de Resultados
RF01 — O príncipe deve se movimentar lateralmente pelo cenário	RF	Pressionar as teclas de direção e observar o deslocamento	Deslocamento contínuo e coerente com a tecla pressionada	A preencher	A preencher
RF02 — O príncipe deve ser capaz de saltar	RF	Pressionar a tecla de salto em solo e em pleno ar	O salto ocorre apenas em solo, com trajetória e retorno consistentes	A preencher	A preencher
RF03 — O cenário deve conter obstáculos (pedras e árvores)	RF	Percorrer um nível registrando os obstáculos renderizados	Pedras e árvores nas posições definidas e visualmente distinguíveis	A preencher	A preencher
RF04 — O jogo deve detectar colisão do príncipe com obstáculos	RF	Conduzir o personagem contra uma pedra e contra uma árvore	Colisão detectada em ambos os casos, acionando a penalidade de vida	A preencher	A preencher
RF05 — Cada nível deve conter exatamente 3 portais	RF	Percorrer o nível do início ao castelo contando os portais	Exatamente 3 portais encontrados no percurso	A preencher	A preencher
RF06 — O contato com um portal deve levar ao mundo de perguntas	RF	Conduzir o personagem até cada um dos 3 portais	Em cada contato o jogo transita de CENÁRIO para PERGUNTAS	A preencher	A preencher
RF07 — Cada portal deve abrir uma questão com alternativas de "a" a "d"	RF	Entrar em um portal e inspecionar a tela exibida	Enunciado e exatamente quatro alternativas rotuladas de "a" a "d"	A preencher	A preencher
RF08 — O jogo deve validar a alternativa escolhida	RF	Responder uma questão de gabarito conhecido, certa e errada	Acerto reconhecido como correto e erro como incorreto, com feedback	A preencher	A preencher
RF09 — O jogador deve iniciar a partida com 3 vidas	RF	Iniciar uma nova partida e inspecionar o HUD	O HUD exibe 3 vidas no início da partida	A preencher	A preencher
RF10 — Errar uma pergunta deve reduzir uma vida	RF	Responder incorretamente e comparar o HUD antes	Contagem de vidas decrementada em exatamente 1 unidade	A preencher	A preencher

		e depois			
RF11 — Colidir com um obstáculo deve reduzir uma vida	RF	Colidir uma única vez com uma pedra e comparar o HUD	Contagem de vidas decrementada em exatamente 1 unidade	A preencher	A preencher
RF12 — Após responder, o jogador retorna ao cenário no ponto do portal	RF	Responder à questão (acerto e erro) e observar a posição	Retorno ao estado CENÁRIO na posição imediatamente após o portal	A preencher	A preencher
RF13 — O jogo é vencido ao alcançar o castelo com ao menos uma vida	RF	Concluir o percurso preservando pelo menos 1 vida	Tela de vitória exibida ao tocar o castelo	A preencher	A preencher
RF14 — O jogo termina em derrota quando as vidas chegam a zero	RF	Provocar perdas sucessivas até zerar as vidas	Partida encerrada e tela de derrota exibida ao chegar a 0 vidas	A preencher	A preencher
RF15 — O HUD deve exibir vidas restantes e feedback de acerto/erro	RF	Observar o HUD durante uma partida com acertos e erros	Vidas e mensagens exibidas e atualizadas em tempo real	A preencher	A preencher
RF16 — O banco não deve repetir questões dentro de um mesmo nível	RF	Jogar um nível completo e registrar as 3 questões sorteadas	As 3 questões apresentadas são distintas entre si	A preencher	A preencher
RNF01 — Usabilidade: controles simples, sem consulta a manual	RNF	Sessão com 3 usuários sem instrução prévia, cronometrada	Todos realizam o primeiro salto em menos de 30 segundos	A preencher	A preencher
RNF02 — Desempenho: taxa de quadros estável	RNF	Medir FPS durante 3 minutos de jogo com contador ativo	Média maior ou igual a 30 FPS, sem quedas perceptíveis	A preencher	A preencher
RNF03 — Portabilidade: executar em máquinas modestas	RNF	Executar o jogo em ao menos duas máquinas distintas da equipe	Jogo inicia e é jogável até o fim do nível em ambas	A preencher	A preencher
RNF04 — Manutenibilidade: código separado por responsabilidade	RNF	Inspeção de código verificando a separação entre os módulos	Cada responsabilidade em módulo próprio, sem duplicação de lógica	A preencher	A preencher
RNF05 — Legibilidade visual: elementos visualmente distinguíveis	RNF	Apresentar capturas a 3 usuários e pedir que identifiquem os elementos	Todos identificam príncipe, obstáculos, portais e castelo	A preencher	A preencher
RNF06 — Modificabilidade: adicionar perguntas sem alterar código	RNF	Inserir nova questão no arquivo do banco e reexecutar o jogo	Nova questão sorteável sem alteração do código-fonte	A preencher	A preencher
RNF07 — Confiabilidade: entradas inválidas não encerram o jogo	RNF	Pressionar teclas fora do conjunto "a"-"d" durante uma questão	Entradas ignoradas, sem travamento e sem perda indevida de vida	A preencher	A preencher

Tabela 1 - Requisitos e Testes Planejados

As evidências de resultados — capturas de tela do jogo em execução, registros de contagem de FPS e fotografias das sessões de teste com usuários — serão inseridas após a Tabela 1 a partir da próxima entrega, numeradas sequencialmente a partir da Figura 5, conforme os testes forem executados.

4. Arquitetura do Sistema

A arquitetura combina dois padrões consagrados de desenvolvimento de jogos. O padrão Game Loop centraliza a atualização e a renderização a cada quadro, e o padrão State faz com que o jogo assuma um estado por vez, atualizando e desenhando apenas o estado ativo. Sobre esses padrões, o sistema divide-se em três camadas: apresentação e jogabilidade, lógica de domínio e dados. A Figura 1 apresenta a organização em blocos, com as interconexões entre os módulos.

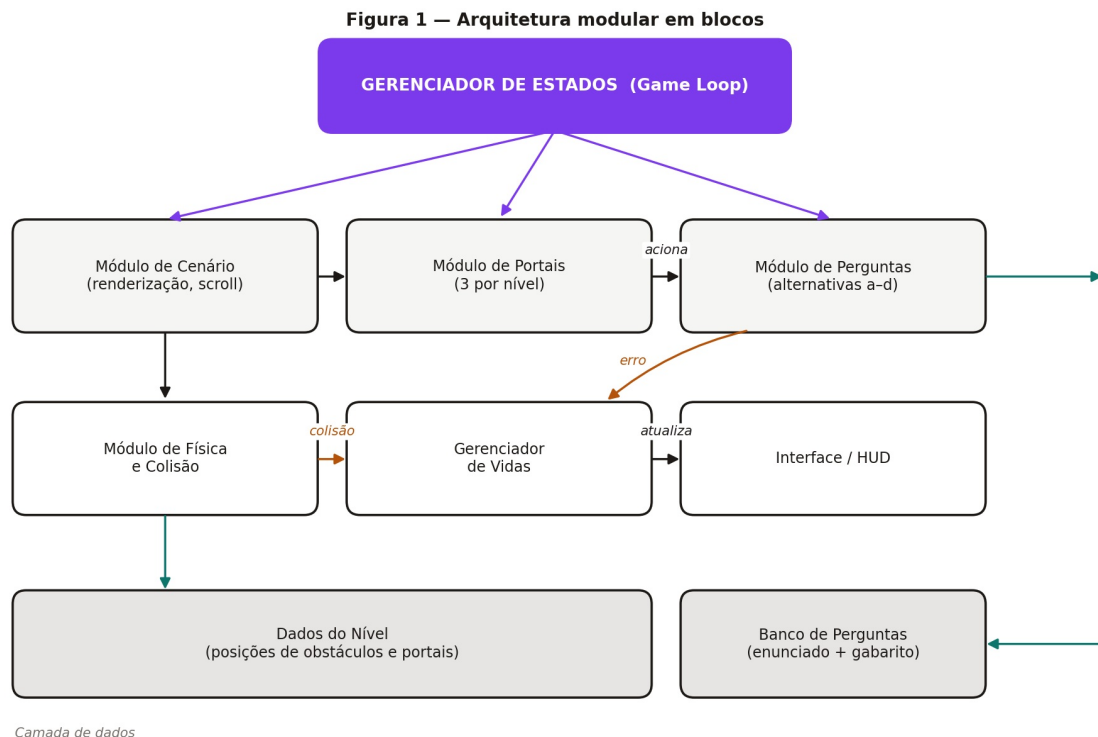


Figura 1 - Arquitetura do sistema em diagrama de blocos

O Gerenciador de Estados é o componente central: controla o laço principal, decide qual estado está ativo e executa as transições. O Módulo de Cenário renderiza o cenário lateral, o príncipe, os obstáculos, os portais e o castelo, controlando o scroll, e atende RF01, RF03 e RNF05. O Módulo de Física e Colisão trata movimentação, salto e detecção de colisão, atendendo RF01, RF02, RF04 e RF11. O Módulo de Portais detecta a entrada do príncipe em um portal e solicita a transição de estado, atendendo RF05, RF06 e RF12. O Módulo de Perguntas solicita uma questão ao banco, exibe enunciado e alternativas e valida a resposta, atendendo RF07, RF08, RF16 e RNF07. O Gerenciador de Vidas mantém a contagem, aplica penalidades e sinaliza a derrota, atendendo RF09, RF10, RF11 e RF14. A Interface exibe vidas, feedback e as telas de vitória e derrota, atendendo RF15, RNF01 e RNF05. Na camada de dados, o Banco de Perguntas e os Dados do Nível residem em arquivos externos ao código, atendendo RF14, RF16, RNF06, RF03 e RF05.

A Figura 2 apresenta a máquina de estados, com os cinco estados do jogo e os eventos que provocam cada transição.

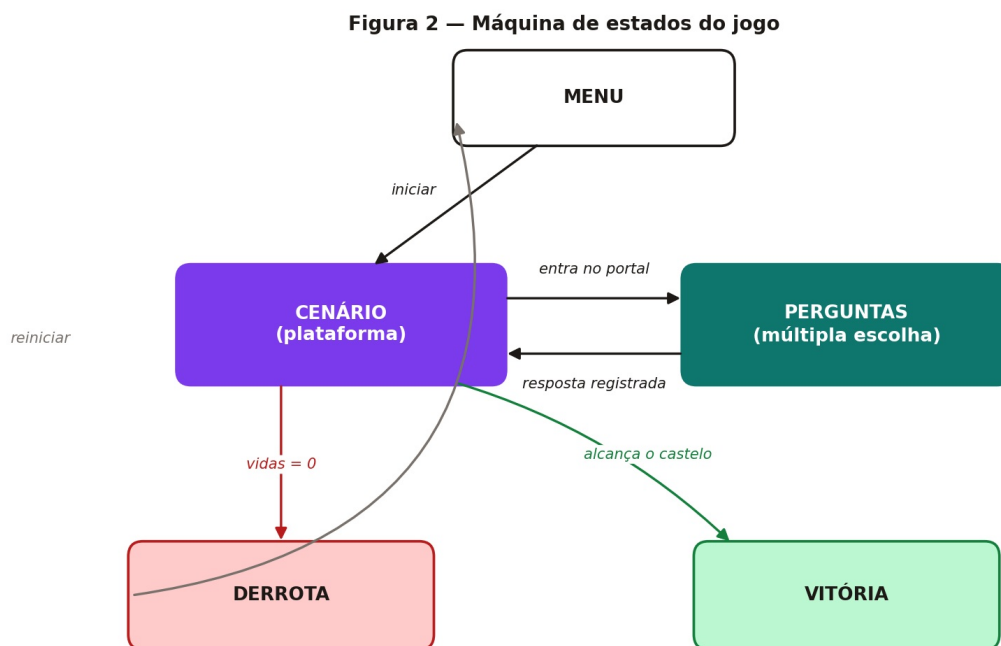


Figura 2 - Máquina de estados do jogo

A verificação de derrota ocorre sempre no estado CENÁRIO. Mesmo quando a vida é perdida por erro em uma pergunta, o jogo primeiro retorna ao cenário e só então avalia a condição de término. A Figura 3 detalha, em diagrama de sequência, a interação entre os componentes quando o jogador entra em um portal e responde incorretamente — o caminho mais crítico do sistema, por envolver todos os módulos de uma só vez.

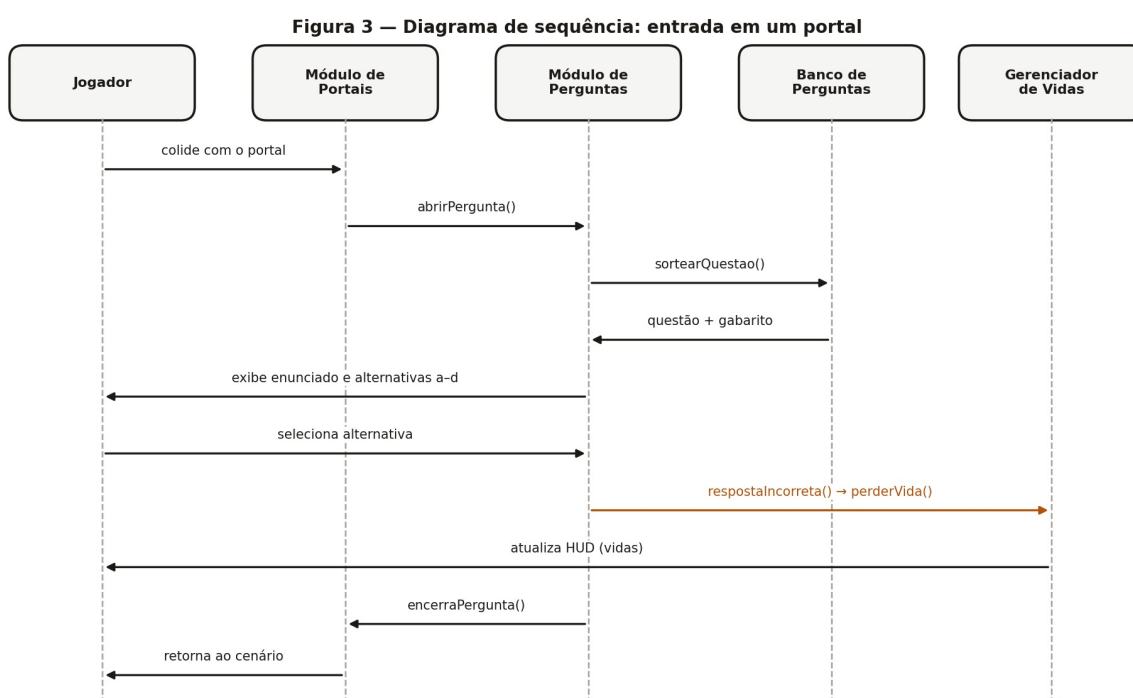


Figura 3 - Diagrama de sequência da entrada em um portal

As decisões arquiteturais justificam-se pelos requisitos estabelecidos na Seção 3. A adoção do padrão State decorre de o jogo alternar entre dois contextos radicalmente diferentes, plataforma e perguntas; separá-los em estados evita condicionais espalhadas pelo laço principal e atende diretamente RNF04. A separação entre o Módulo de Portais e o Módulo de Perguntas decorre de o portal ser um elemento do cenário enquanto a pergunta é conteúdo, o que permite alterar a quantidade ou o posicionamento dos portais sem tocar na lógica das questões. Manter o Banco de Perguntas em arquivo externo atende RNF06, pois novas questões passam a ser adicionadas editando um arquivo de dados, sem recompilação, o que também facilita a revisão do conteúdo por integrantes não responsáveis pelo código. Manter os Dados do Nível igualmente externos permite ajustar a dificuldade durante os testes sem alterar código, acelerando o ciclo de depuração da Seção 5. Concentrar a penalidade no Gerenciador de Vidas decorre de colisão e resposta incorreta serem causas diferentes com o mesmo efeito, o que garante consistência e um único ponto de verificação da derrota. Restringir a verificação de derrota ao estado CENÁRIO simplifica a máquina de estados, pois o estado PERGUNTAS passa a ter uma única saída, reduzindo o número de transições e de casos de teste. Por fim, a renderização 2D com sprites simples atende RNF02 e RNF03, dispensando aceleração gráfica dedicada.

A escolha definitiva de linguagem e biblioteca gráfica será consolidada na próxima etapa. As candidatas em avaliação são bibliotecas de jogos 2D com suporte nativo a sprites, laço principal e captura de eventos de teclado, priorizando baixo custo computacional e ausência de dependências de difícil instalação, conforme RNF02 e RNF03.

5. Método Experimental

O desenvolvimento segue um ciclo iterativo e incremental, no qual cada módulo da arquitetura é planejado, implementado, verificado contra os testes da Tabela 1 e integrado ao repositório. A Figura 4 ilustra o método adotado.

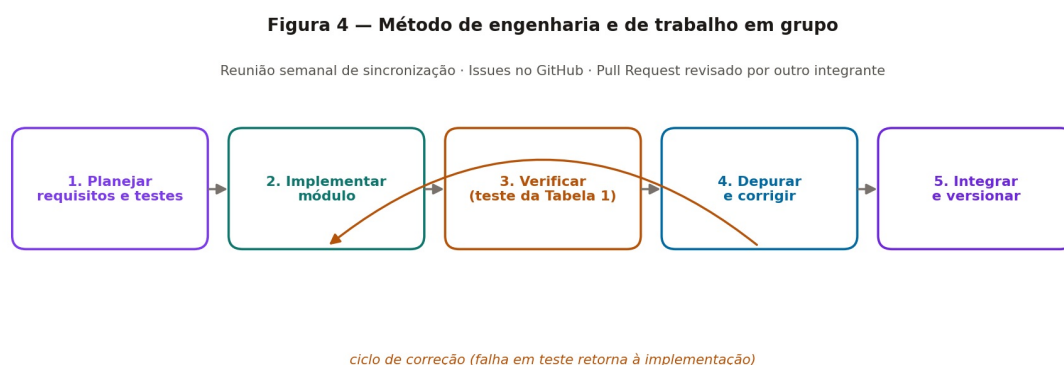


Figura 4 - Método de engenharia e método de trabalho em grupo

Na etapa de planejamento, identificam-se para cada módulo os requisitos da Tabela 1 que ele deve satisfazer e os testes correspondentes; nenhuma implementação começa sem que o critério de aceitação esteja escrito. Na implementação, o módulo é codificado isoladamente, respeitando as fronteiras de responsabilidade definidas na Seção 4. Na verificação, executam-se os testes associados ao requisito: os testes de comportamento são executados manualmente com roteiro fixo e os de desempenho usam contador de FPS em tela. Em caso de falha, aplica-se o procedimento de depuração descrito adiante e retorna-se à implementação. Concluída a verificação, abre-se um Pull Request, obtém-se a revisão de outro integrante e registra-se a mudança no CHANGELOG.

As ferramentas utilizadas são Git e GitHub para controle de versão e colaboração, com branches, Issues, Pull Requests e Releases; Markdown no repositório para a documentação, compilada em PDF para entrega no Moodle; editor de código com depurador integrado para o desenvolvimento; contador de FPS embutido no próprio jogo, em modo de depuração, para a verificação de desempenho; e capturas de tela e gravações da execução para o registro de evidências.

A verificação organiza-se em três níveis. A verificação de módulo exercita cada módulo isoladamente assim que concluído, como no caso do Módulo de Perguntas, testado com um banco reduzido antes de existir cenário. A verificação de integração testa as transições entre estados, especialmente o par crítico CENÁRIO e PERGUNTAS detalhado na Figura 3. A verificação de sistema consiste em uma partida completa do menu até vitória ou derrota, percorrendo os três portais, e valida RF13, RF14 e os requisitos não-funcionais. Cada execução de teste preenche uma linha da Tabela 1, de modo que a tabela funciona simultaneamente como especificação e como registro de verificação.

Diante de uma falha, o grupo adota quatro passos. Primeiro, reproduzir: determinar a sequência mínima de ações que provoca a falha e registrá-la como Issue no GitHub. Segundo, isolar: identificar em qual módulo da Figura 1 a falha se manifesta, usando o diagrama para delimitar as fronteiras suspeitas. Terceiro, instrumentar: inserir registros em tela ou em console para os valores críticos, como posição do personagem, estado ativo, contagem de vidas e índice da questão sorteada. Quarto, corrigir e reverter: aplicar a correção e reexecutar não apenas o teste que falhou, mas todos os testes do módulo afetado, evitando regressões. Falhas antecipadas como prováveis, dado o desenho do sistema, são a detecção de colisão imprecisa nas bordas dos sprites, o disparo repetido do mesmo portal em quadros consecutivos e a perda de vida indevida por entrada de teclado registrada duas vezes na tela de perguntas, coberta pelo teste de RNF07.

O método de trabalho em grupo apoia-se na divisão por módulo, já que a arquitetura em blocos da Figura 1 define naturalmente as frentes de trabalho e permite desenvolvimento paralelo com baixo acoplamento e poucos conflitos de integração. Realiza-se uma reunião semanal de sincronização para revisar o que foi concluído, o

que está bloqueado e o replanejamento da semana seguinte. Cada tarefa e cada falha viram uma Issue vinculada ao requisito correspondente da Tabela 1, garantindo rastreabilidade entre requisito, código e teste. Nenhum Pull Request é integrado sem revisão de um integrante que não escreveu o código, o que difunde o conhecimento do sistema por todo o grupo. Cada entrega semanal gera uma Release no GitHub, com o CHANGELOG atualizado.

6. Lições Aprendidas

Nesta primeira semana, dedicada ao planejamento, à documentação e à organização do repositório, o principal problema enfrentado foi a vagueza dos requisitos herdados dos relatórios anteriores. A redação inicial continha itens como "o jogo deve ter um sistema de vidas", descrição que não permite construir um caso de teste objetivo. Ao preencher a coluna Teste da Tabela 1, o grupo percebeu que vários requisitos precisavam ser reescritos com valores concretos, como três vidas iniciais, três portais por nível e decremento de exatamente uma unidade. O aprendizado é que escrever o teste junto com o requisito funciona como o melhor filtro de qualidade da especificação: requisito que não gera teste claro é requisito mal escrito.

O segundo desafio foi definir as fronteiras de responsabilidade entre módulos, especialmente para decidir onde alocar a verificação de derrota: no Gerenciador de Vidas, no Módulo de Perguntas ou no estado CENÁRIO. Cada alternativa foi discutida quanto ao número de transições resultantes na máquina de estados. Concluiu-se que decisões arquiteturais devem ser avaliadas pelo que simplificam, e não apenas pelo que permitem, já que concentrar a verificação em um único estado reduziu tanto o código quanto a quantidade de casos de teste necessários. Esse debate só se tornou possível ao desenhar a máquina de estados da Figura 2, o que mostrou que o diagrama funciona como instrumento de descoberta de decisões, e não apenas de comunicação.

Houve ainda o aprendizado do fluxo de Releases do GitHub e da convenção de versionamento semântico, além da definição de uma estrutura de pastas que sobrevivesse ao crescimento do projeto. Definir essa estrutura e o padrão de commits antes do início da codificação evitou retrabalho de reorganização e manteve o histórico legível desde a primeira Release.

Para as próximas semanas, o grupo adotará os seguintes refinamentos. As evidências de teste serão registradas no momento da execução, e não ao final da semana, evitando reexecuções. Toda Issue será vinculada ao identificador do requisito correspondente da Tabela 1, garantindo rastreabilidade entre requisito, código e teste. O contador de FPS será implementado já no primeiro protótipo jogável, para que RNF02 possa ser medido continuamente. E os arquivos Markdown serão atualizados no mesmo Pull Request que altera o código, evitando defasagem entre documento e sistema.